Kru — Project Plan & Milestones

Nov 1, 2026 · @Amin Haiqal

Overview

Kru is a headless, local-first engine that runs AI agent CLIs (Claude Code, Codex, Gemini, Cursor and others) as persistent teammates. Every agent gets a durable mailbox, a workspace and a thread that survives provider switches and crashes. Work is delivered one turn at a time, and agents message each other through the same mailbox.

Pitch: *Give every agent a mailbox. Kru delivers the work and never loses a message.*

The name: *kru* is Malay for "crew" — a crew of agents that hand work to each other. One syllable, easy to say, easy to type as a command.

Who uses it

- Developers who want several agent CLIs to work on one codebase, each with its own workspace and memory.
- Teams that put agents behind their own tools: a Slack bot, a CI job, an internal dashboard.
- Builders who need an engine they can embed, not a desktop app.

How it differs from OpenBot

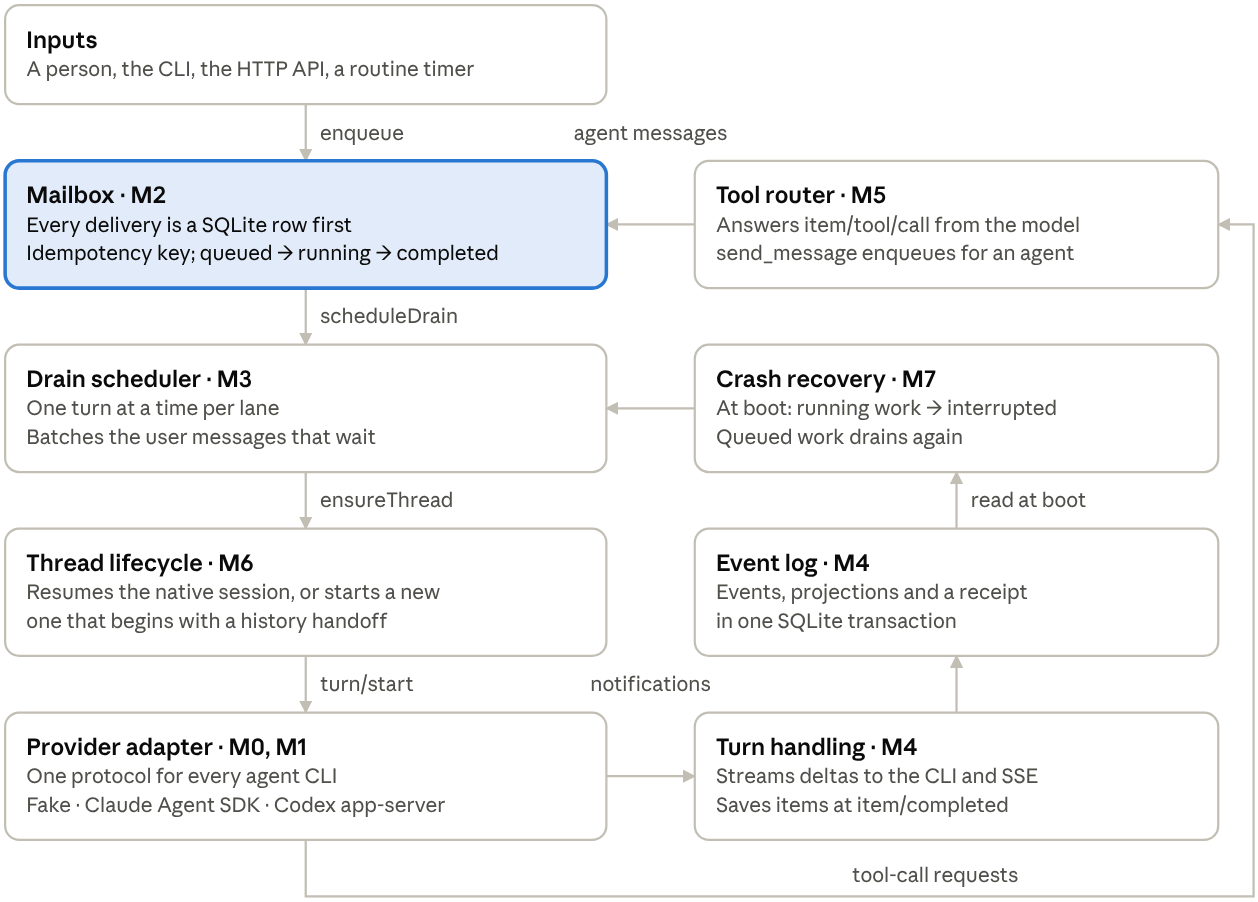
	OpenBot	Kru
Form	Electron desktop app	Daemon, CLI, HTTP + SSE API, TypeScript library
Audience	End users	Developers and integrators
Scope	Browser, mobile, voice, team servers	The core engine only
License	PolyForm Noncommercial 1.0.0	MIT (allows commercial use)
Code	—	Written from scratch from the ideas; no copied code

The license row is why the last row matters: OpenBot's code cannot go into an MIT project, so study a file, close it, then write your own version.

Architecture

Work flows down the left column and results flow back up the right. Agent messages loop back into the same mailbox, so there is only one execution path.

Every piece of work enters through the mailbox



Kru message lifecycle · 9 parts, labelled by milestone

Core concepts

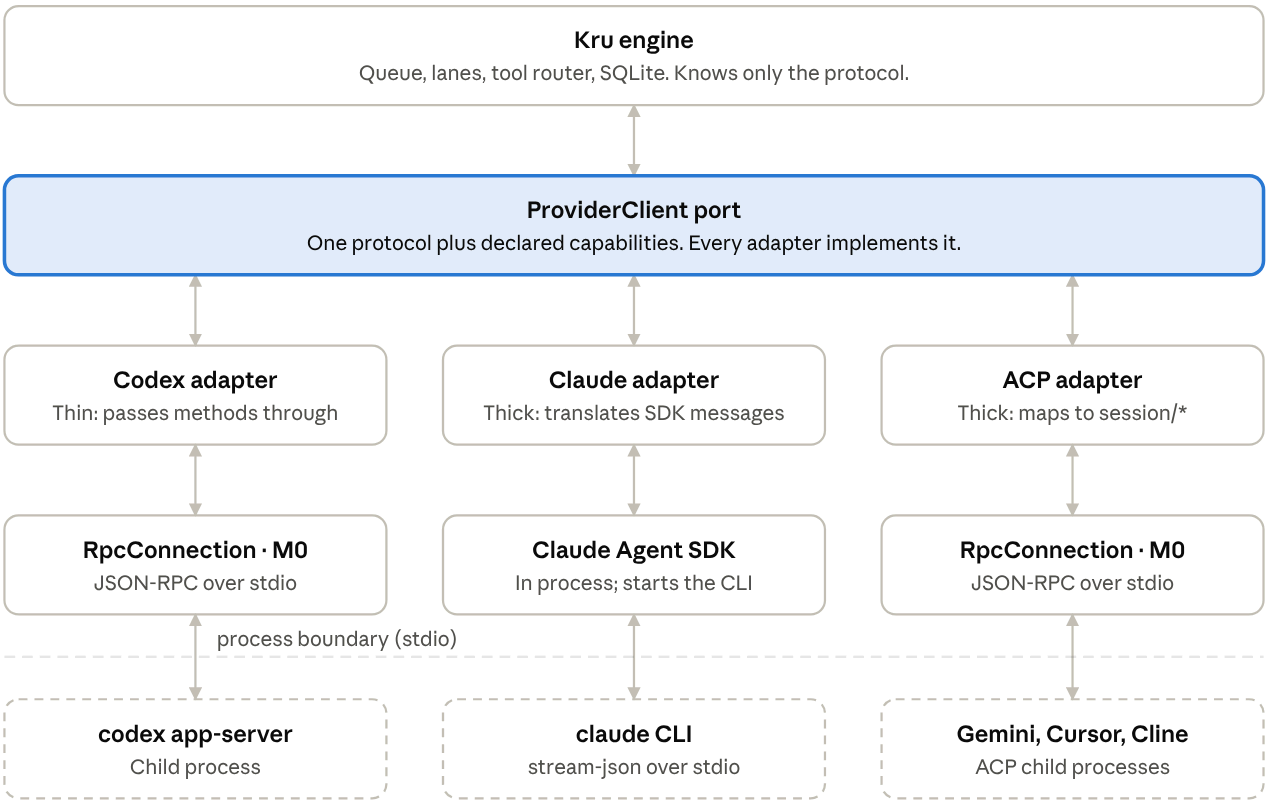
- **Delivery:** one unit of work for one agent, stored before anything runs.
- **Lane:** a queue that runs one turn at a time — an agent's main chat, or one of its side threads.

- **Thread vs provider session:** the thread is Kru's and permanent; the session is the CLI's and disposable.
- **Handoff:** a size-limited summary of recent history, prepended to the first turn of a new session.
- **Turn:** one `turn/start` to `turn/completed` exchange with a provider.

Provider layers: ports and adapters

Kru's engine depends on one interface, never on a specific agent. This is hexagonal architecture, also called ports and adapters: the `ProviderClient` interface is the port, and each agent CLI plugs in through its own adapter. Adding an agent means adding one adapter; the engine does not change.

The engine talks to one port; adapters absorb each agent difference



Kru provider layers · engine, port, 3 adapters, transports, agent processes

Arrows run both ways: commands go down, events and tool-call requests come up. Dashed boxes are separate processes that Kru starts and can lose at any time.

What each layer owns

Layer	Owns	Knows about	Milestone
Engine	What runs and when: queue, lanes, tool router, SQLite	The protocol only	M2–M7
`ProviderClient` port	The contract: protocol methods and declared capabilities	Nothing below it	M1
Adapters	Translation between the protocol and one agent's native API	One agent	M1
Transport	Framed messages over stdio, request ids, timeouts	Bytes and JSON-RPC	M0
Agent process	The model, its tools, its own session files	Nothing about Kru	External

The shared language. OpenBot uses the Codex App Server protocol as the language every adapter speaks, because Codex already had a clean JSON-RPC design. Kru keeps the same method names (see Core types) but gives each method's params and result a TypeScript type, so the compiler checks every adapter. JSON-RPC framing stays inside the transport.

How each adapter translates

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`thread/start`	Sent as-is	Creates an SDK session itself	`session/new`
`turn/start`	Sent as-is	Calls the SDK's `query()` with the prompt	`session/prompt`
`item/agentMessage/delta` (up)	Received as-is	Built from SDK assistant messages	Built from `session/update` (`agent_message_chunk`)

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`turn/completed` (up)	Received as-is	Built when the SDK result message arrives	Built when `session/prompt` returns
`item/tool/call` (up, request)	Received as-is	Built from the SDK tool callback	Built from ACP tool requests

The Codex adapter is thin. The Claude and ACP adapters are thick — about 2,400 and 3,150 lines in OpenBot — because they act as a JSON-RPC server: they answer requests themselves and build notifications from what their agent emits.

One turn through the layers (Claude)

1. The engine calls `turn/start` with the thread id and the prompt.
2. The Claude adapter calls `query()` and answers at once with the new turn id.
3. The SDK writes to the `claude` CLI's stdin and reads its stdout as stream-json.
4. Text arrives; the adapter emits `item/agentMessage/delta` notifications up to the engine.
5. The model calls `send_message`; the adapter sends an `item/tool/call` request up and waits for the engine's answer.
6. The SDK reports the result; the adapter emits `turn/completed`.

The engine sees the same six events it would see from Codex.

Where state lives

Layer	Keeps	When it is lost
Transport	Pending requests by id	Rejected on process exit (M0)
Adapter	Native session ids, the active turn	Session resumed or replaced with a handoff (M6)
Engine	Threads, deliveries, messages in SQLite	Never: SQLite is the source of truth

Rules that keep the layers clean

1. Dependencies point inward: the engine imports the port, never an adapter. Adapters are chosen in one registry.
2. An adapter never writes to SQLite or to the screen. It only emits protocol events.
3. The transport knows nothing about turns, threads or tools.
4. Each adapter declares a `capabilities` object (compaction mode, steering, subagents). The engine uses a feature when it is present, instead of reducing every agent to the lowest common denominator.
5. Every adapter passes the same conformance test suite: the fake, Codex and Claude run the same file.

Why it pays off

- **One file per agent.** A new provider is a new adapter; nothing else changes.
- **Free tests.** The fake agent is just another adapter, so M2–M7 run with no model and no API key.
- **Contained failures.** A crashed CLI becomes one `exit` event; the engine marks the turn interrupted and nothing above needs to know why.
- **Provider switching.** The thread never belonged to Codex or Claude, so it can move between them (M6).
- **Comparable results.** Every agent looks the same to the engine, so the same task can run on each and be measured — the base for evals and cross-provider review after v0.1.

Stack, ground rules and layout

Use TypeScript on Bun with `bun:sqlite`, the same stack as OpenBot, so you can compare files side by side. Use plain `async` / `await`; learn Effect later as a separate project.

Ground rules

1. Build the fake provider first. Every test runs against it: deterministic, free, and able to stall or crash on command. OpenBot does the same in `scripts/provider-fakes/`.
2. Write your version before you read theirs. Then read their file and note what they handled that you missed.
3. Every milestone ends with a test that proves its "Done when" line, and a small user-facing result.
4. SQLite is the source of truth. Provider session files are cache.

5. Never retry a turn whose outcome is unknown. Mark it `interrupted` and let a person decide.

Repository layout

```
kru/
  src/
    rpc/          JSON-RPC over stdio (M0)
    providers/    ProviderClient + adapters: fake, claude, codex (M1)
    store/        SQLite, events, projections, migrations (M2, M4)
    mailbox/      enqueue, delivery state machine (M2)
    drain/        lanes and scheduler (M3)
    turns/        notification handling, delta buffer (M4)
    tools/        tool router, send_message, history tools (M5, M6)
    threads/      provider sessions, handoff (M6)
    boot/         crash recovery (M7)
    api/          HTTP + SSE server (v1.0)
    cli/          kru command (every milestone)
  test/
    fakes/        scripted fake agents: echo, stall, crash, tool-caller
  examples/
  docs/
```

Core types

Five types carry the whole engine. Start with these and change them as the milestones teach you more.

```
// Who sent a delivery. Users, agents and routines share one queue.
type Sender =
  | { kind: "user" }
  | { kind: "agent"; agentId: string }
  | { kind: "routine"; routineId: string };

type DeliveryStatus = "queued" | "starting" | "running" | "completed" |
"failed" | "interrupted";

// One unit of work for one agent. A row in SQLite, never only in memory.
interface Delivery {
  id: string;
  recipientAgentId: string;
  laneId: string | null;          // null = the agent's main chat
```

```
  sender: Sender;
  text: string;
  replyToMessageId: string | null;
  expectsReply: boolean;
  status: DeliveryStatus;
  idempotencyKey: string | null;  // UNIQUE: a retry returns the first
receipt
  createdAt: string;
}

// Kru's thread. Survives provider switches.
interface Thread { id: string; agentId: string; laneId: string | null }

// The provider's private session, bound to one thread. Disposable.
interface ProviderSession { threadId: string; provider: string;
externalSessionId: string; retiredAt: string | null }

// Every provider adapter implements this. One protocol for all of them.
interface ProviderClient {
  readonly provider: string;
  start(): Promise<void>;
  stop(): Promise<void>;
  request<T>(method: string, params: unknown, timeoutMs?: number):
Promise<T>;
  respond(id: string | number, result: unknown): void;
  respondError(id: string | number, error: { code: number; message: string
}): void;
  on(event: "notification", fn: (n: { method: string; params: any }) =>
void): void;
  on(event: "request", fn: (r: { id: string | number; method: string; params:
any }) => void): void;
  on(event: "exit", fn: (error: Error) => void): void;
}
```

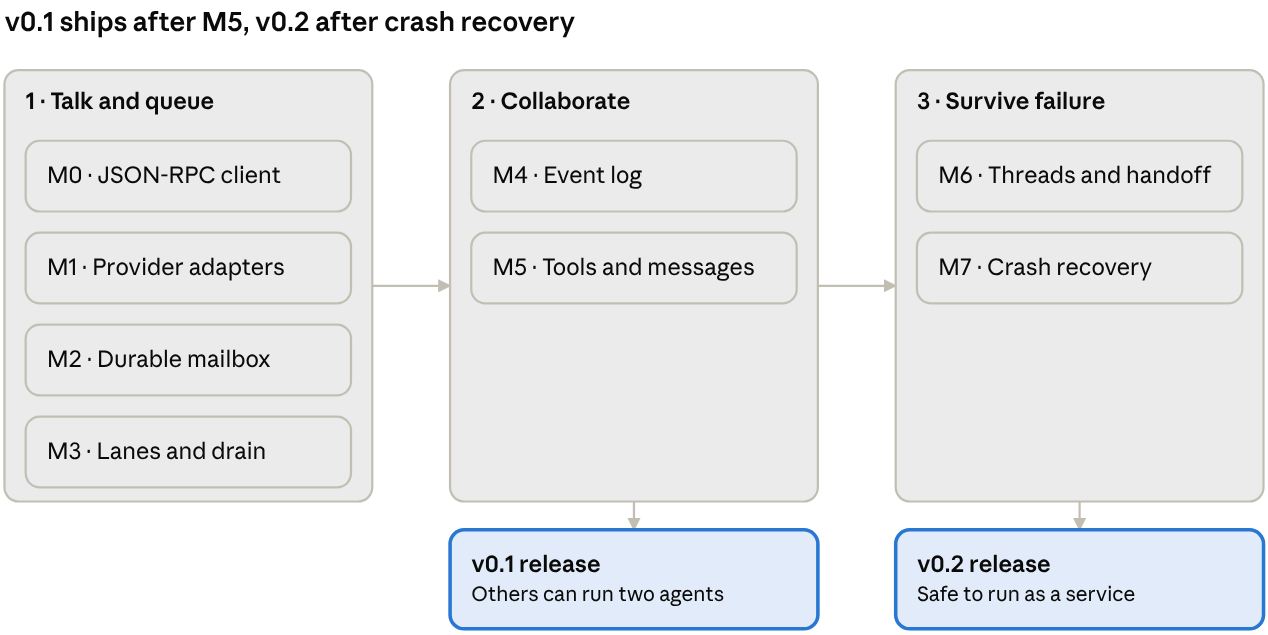
The protocol methods every adapter must speak:

Direction	Method	Meaning
Kru → provider	`thread/start`, `thread/resume`	Open or reopen a native session
Kru → provider	`turn/start`, `turn/interrupt`	Run one turn, or stop it

Direction	Method	Meaning
Provider → Kru	`turn/started`, `turn/completed`	Turn boundaries, with status
Provider → Kru	`item/started`, `item/agentMessage/delta`, `item/completed`	Streamed output and tool steps
Provider → Kru	`thread/tokenUsage/updated`	Context fill, for compaction
Provider → Kru (request)	`item/tool/call`	The model calls a Kru tool and waits for the answer

Milestones

Eight milestones, each isolating one idea. After M5 Kru is usable by other people (v0.1). After M7 it is crash-safe (v0.2). Each milestone lists what to build, which OpenBot files to read after you build it, the test that proves it, and what a user can do with it.



Kru roadmap · 8 milestones in 3 phases, 2 releases

Each phase depends on the one before it; inside a phase, build in order.

M0 — JSON-RPC over stdio (the transport)

Purpose. Kru does not run a model. It starts agent CLIs as child processes and talks to them over stdin and stdout. M0 builds the bottom layer of the hexagon: a generic connection that sends and receives JSON-RPC messages reliably. It knows nothing about Codex, Claude, turns or threads, and it is proven against a fake agent. In M1 the Codex and ACP adapters use it directly; the Claude adapter uses the Agent SDK instead.

Public API

```
interface RpcConnectionOptions {
  command: string;
  args: string[];
  env?: Record<string, string>;
  requestTimeoutMs: number;
  maxLineBytes: number;        // one message = one line; longer = fatal
  maxQueuedWriteBytes: number; // stdin backpressure cap
  maxBadLines: number;         // consecutive unparsable lines before failing
}

class RpcConnection {
  constructor(options: RpcConnectionOptions);
  start(): void;
  stop(): Promise<void>;       // ends the whole process tree
  request<T>(method: string, params: unknown,
    options?: { timeoutMs?: number; signal?: AbortSignal; decode?: (value:
unknown) => T }): Promise<T>;
  notify(method: string, params?: unknown): Promise<void>;
  // The transport sends the answer: the resolved value, or an error if it
throws.
  onRequest(handler: (request: { id: string | number; method: string; params:
unknown; signal: AbortSignal }) => Promise<unknown>): void;
  on(event: "notification", listener: (n: { method: string; params: unknown
}) => void): void;
  on(event: "diagnostic", listener: (d: Diagnostic) => void): void; // stderr
| bad-line | late-response | write-blocked
  on(event: "exit", listener: (error: ProcessExitError) => void): void; //
fires exactly once
}
```

Design rules

1. **Pure transport.** No method names, turns or threads inside this module. Agent-specific behaviour, such as a liveness probe that needs a cheap method, belongs to the M1 adapter.
2. **Frame on bytes.** Split on the newline byte before decoding UTF-8, so a character split across two chunks is never corrupted.
3. **Every request gets exactly one answer.** The handler's value becomes the result; a thrown error becomes `-32603`; no handler becomes `-32601`. A deadlock from a missing listener is impossible.
4. **A bad line is a diagnostic, not a crash.** Messages before and after it in the same chunk are still handled. After `maxBadLines` consecutive bad lines the connection fails, so a truly broken CLI is still restarted.
5. **Backpressure.** When `stdin.write()` returns false, wait for `drain`. Queued bytes above `maxQueuedWriteBytes` reject the write with a clear error instead of growing memory.
6. **Late answers are visible.** Remember recent timed-out ids in a bounded set. An answer that arrives later becomes a `late-response` diagnostic with the method and the delay.
7. **Stale-process guard.** Events from an old child are ignored after a restart, and request ids never reset, so an old answer cannot resolve a new request.
8. **Exit reports the real reason.** On exit, wait briefly for stderr to end, then reject every pending request with a `ProcessExitError` that carries the last stderr line.
9. **Stop ends the whole tree.** Terminate the child and its children; force-kill after a grace period, so no MCP servers are left behind.
10. **One line has a size limit.** Past `maxLineBytes` the stream cannot recover, so the connection fails. OpenBot sets 256 MiB, because one Codex thread resume reached 21 MB.

Fake agent (`test/fakes/fake-agent.ts`) — a script that speaks the protocol and, on command: answers normally, answers out of order, stalls, crashes, sends notifications, sends a request back, writes split, merged or invalid lines, writes to stderr, and stops reading stdin. Every later milestone reuses it.

Tests — `bun test`, no network, no API key, about 2 seconds.

#	Test	Proves
1	`ping` returns `pong`	Basic round trip

#	Test	Proves
2	100 concurrent requests answered in random order	Each answer reaches its own caller
3	Error response	Rejects with the code and message
4	Fake stalls	Rejects with a timeout error
5	Five notifications	Arrive in order
6	Fake sends a request; the handler answers	The fake receives the answer and echoes it back
7	One message split across two chunks; two messages in one chunk	Framing is correct
8	A bad line between two good lines in one chunk	Both good messages handled; one `bad-line` diagnostic; connection stays up
9	Fake exits during a request	Every pending request rejects; `exit` fires once, with the last stderr line
10	`stop()`, then `request()`	Child tree is gone; the new request rejects at once
11	Fake writes to stderr	Reported as a diagnostic, never parsed as protocol
12	Fake stops reading; 50 MB of writes	Memory stays bounded; writes wait or fail cleanly
13	Timeout, then the fake answers late	One `late-response` diagnostic; no crash, no double resolve
14	Fake sends a request with no handler set	It receives a `-32601` error

After every test: the pending map is empty, no child process is running, and there are no unhandled promise rejections.

Done when: all 14 tests and the hygiene checks pass, every time.

You can now:

```
$ kru rpc ping --fake
pong (1.4 ms)

$ kru rpc ping --fake --stall
error: request "ping" timed out after 2000 ms
```

Stretch: send `initialize` to a real `codex app-server --listen stdio://` and print the answer.

Read after: `src/backend/app-server-client.ts`, `src/backend/jsonl.ts`, `src/backend/protocol.ts`.

Out of scope: providers, SQLite, queues, retries.

Improvements over OpenBot

OpenBot's transport is production-hardened. Kru keeps its byte framing, line limit, stale-process guard, stderr-before-exit and process-tree kill, and changes six things:

Area	OpenBot	Kru M0
Separation	The Codex client mixes transport with Codex logic (turn interrupts, resume params, history paging); ACP and Pi have their own transports	One generic transport for every JSON-RPC agent; agent logic lives in adapters
Bad line	Any unparsable line ends the process and every agent on it; valid messages earlier in the chunk are dropped	Diagnostic and skip; fail only after `maxBadLines` in a row
Backpressure	`stdin.write()` result ignored; writes queue without limit	Wait for `drain`; cap queued bytes
Late answers	Ignored silently after a timeout	Reported as `late-response`, so the engine can reconcile a turn that did start
Unanswered requests	The tool router answers unknown methods, but the transport does not guarantee an answer	The transport always answers exactly once

Area	OpenBot	Kru M0
Types	`params: unknown`, checked deep in the engine	Decoded at the boundary through `decode`

One trade-off: OpenBot's fail-fast on a bad line is a deliberate safety choice. Skipping is safe here only because each NDJSON line is independent, which is why the `maxBadLines` limit stays.

M1 — One protocol, many providers

- **Build:** the `ProviderClient` interface, a fake adapter, and one real adapter: `codex app-server --listen stdio://` or the Claude Agent SDK. Translate each into the protocol table above.
- **Read after:** `src/backend/agent-client.ts`, `src/backend/claude-client.ts` (the method switch near line 494 and the functions that emit `item/*`), `src/backend/provider-drivers.ts`.
- **Done when:** the same chat works with `--provider fake` and `--provider claude`, and no other code changes.
- **Watch for:** providers that stream text before they announce the item. Create the message on the first delta.
- **You can now:** `kru chat --provider claude` streams a reply in the terminal.

M2 — The durable mailbox

- **Build:** a `deliveries` table and the status state machine. `enqueue()` takes an idempotency key with a `UNIQUE` index and returns the first receipt on a retry.
- **Read after:** `src/backend/mailbox-store.ts`.
- **Done when:** the same key sent twice gives one row; queued rows survive a restart; an illegal transition (`completed → running`) is refused.
- **You can now:** `kru send <agent> "text"` and `kru queue <agent>`.

M3 — Lanes and the drain scheduler

This is the most important milestone. The scheduler decides what runs, when, and in which order.

- **Build:** `scheduleDrain(agentId)` deduplicated by a "scheduled" set and a "draining" set. One turn at a time per lane (an agent's main chat, or one side thread). After a turn ends, schedule the lane again. Batch consecutive user messages into one prompt.
- **Read after:** `src/backend/agent/drain-scheduler.ts` — `#scheduleLane`, `#drainLane`, `startDelivery`.
- **Done when:** five fast messages give one running turn, then one batched turn for the other four; two agents run in parallel; no turn starts twice.
- **Watch for:** the lost wakeup. A message that arrives while a drain finishes is never run. OpenBot's `Effect.ensuring` block checks `nextQueued` again after it clears the draining flag.
- **You can now:** `kru daemon` runs agents in the background while you queue work.

M4 — Event log and projections

- **Build:** `dispatch(commandId, events, project)` in one `BEGIN IMMEDIATE` transaction, with a command-receipts table. Stream deltas to listeners from memory; persist a message at `item/completed` or on a buffered flush.
- **Read after:** `src/backend/database/database-core.ts` (`dispatch`), `src/backend/agent/delta-buffer.ts`, `handleNotification` in `src/backend/agent/turn-lifecycle.ts`.
- **Done when:** replaying the events table rebuilds identical projections; a command run twice returns its stored result.
- **Watch for:** a nested `BEGIN`. Open a transaction only when none is open.
- **You can now:** `kru log <agent>` shows full history after a restart.

M5 — Tools and agent-to-agent messages

- **Build:** route the provider's `item/tool/call` requests. Implement `send_message` as `mailbox.enqueue({ sender: { kind: "agent" } })` with the key `threadId + turnId + callId`. Add `list_agents`.
- **Read after:** `handle` and `#sendMessage` in `src/backend/agent/openbot-tool-router.ts`, `src/backend/agent/tool-call-idempotency.ts`.
- **Done when:** agent A asks agent B, A's turn ends without waiting, and B's answer reaches A as a new delivery. Then cause an acknowledgement loop on purpose (A thanks B, B thanks A) and stop it with `expectsReply: false`.

- **You can now:** create a reviewer agent and a coder agent that hand work to each other. Release **v0.1**.

M6 — Threads apart from provider sessions

- **Build:** your `threads` table and a `provider_sessions` binding table. On a provider switch or a missing session, start a new native session and prepend a handoff packet under a character budget. Add a `history_search` tool for what the handoff cut.
- **Read after:** `ensureThread` in `src/backend/agent/thread-lifecycle.ts`, `src/backend/agent/handoff-history.ts`, "History retrieval" in `docs/architecture/providers.md`.
- **Done when:** after ten turns on Claude you switch the agent to Codex, and it still answers questions about the first turn.
- **You can now:** `kru agent set <agent> --provider codex` with no lost context.

M7 — Crash recovery

- **Build:** at boot, before providers start, mark every `starting` or `running` delivery and every streaming message `interrupted`. Never retry them automatically. Let queued work continue.
- **Read after:** `src/backend/agent/boot-recovery.ts`.
- **Done when:** you `kill -9` the daemon during a turn of the stalling fake, start it again, and the state is consistent and the queue continues.
- **You can now:** run Kru as a service and trust it. Release **v0.2**.

Road to v1.0

After M7 the engine is complete. v1.0 adds what makes Kru useful to people who never read its code: an API, scheduled work, and limits that keep it safe on a shared machine.

Feature	What it adds	Read in OpenBot
HTTP + SSE API	Send, list, stream and cancel over HTTP, so any tool can drive Kru	`src/main/team-api-server.ts` (shape only)

Feature	What it adds	Read in OpenBot
Routines	A timer or webhook that enqueues a delivery; no new execution path	`src/backend/agent/routine-scheduler.ts`, `docs/architecture/events.md`
Context compaction	Ask the provider to compact near 80% of the window; rotate the session when it cannot	`src/backend/agent/context-compaction.ts`
Global turn slots	A cap on concurrent turns, people's messages first	`src/backend/agent/turn-slots.ts`
Deferred tool discovery	`tool_search` and `tool_call` keep the always-loaded tool list small	`src/backend/agent/tool-catalog.ts`
Agent memory	`remember`, `search_memories`, `forget_memory` tools on a SQLite table	`src/backend/agent/agent-memories.ts`
Workspace sandbox	Each agent writes only to its own folder plus allowed roots	`src/backend/agent/workspace-sandbox.ts`
Usage limits	Hold a lane when the provider reports a limit; resume when it resets	`src/backend/agent/usage-limit-gate.ts`

Build the API first. It turns Kru from a CLI into a platform, and every later feature can be tested through it.

Release checklist

People adopt a tool they can install and see working in five minutes. Do these before you announce v0.1.

- [ ] Check that the name "kru" is free on npm and GitHub; short names are often taken, so a scoped package such as @<your-name>/kru is the fallback
- [ ] Add an MIT `LICENSE` and a note that the project is inspired by OpenBot and shares no code with it
- [ ] One-line install: `bunx kru` or `npm i -g kru`

- [ ] README with a 30-second demo GIF: two agents hand work to each other
- [ ] A quick start that works with the fake provider, so a reader needs no API key to try it
- [ ] `examples/` : coder + reviewer pair, a Slack bot over the HTTP API, a nightly routine
- [ ] `docs/architecture.md` with the lifecycle diagram from this doc
- [ ] CI that runs every test against the fake providers
- [ ] A security note: agents run with the provider's own permissions; Kru adds no sandbox until v1.0
- [ ] Secrets are redacted from logs and stored error messages
- [ ] A `CHANGELOG.md` and semantic version tags
- [ ] A write-up of what you learned at each milestone, for your portfolio

How to read the OpenBot source

Read the docs for *why*, the tests for *what*, and the history for *how they got there*. File paths in this doc are from nightly-labs/openbot at commit `0b311bf` (v0.57.0).

1. Clone the full history, not a shallow copy: `git clone https://github.com/nightly-labs/openbot.git` .
2. Read `docs/architecture/structure.md` , `providers.md` and `src/backend/AGENTS.md` before any code. They state the rules the code obeys.
3. Read `docs/glossary.md` . It maps product words (thread, side thread, delivery, lane) to code names.
4. Treat `scripts/*-e2e.ts` and `scripts/provider-fakes/agents/*.ts` as behaviour specs. `stall.ts` and `session-errors.ts` show the failures they guard against.
5. Use `git log -p -- <file>` on the file of your current milestone. Bug fixes show the edge cases you will hit.
6. Follow one message by searching method names: `enqueue` → `scheduleDrain` → `startDelivery` → `"turn/start"` → `handleNotification` → `completeTurn` .
7. Skip `src/renderer` , `src/main` and `apps/` at first. They are product surface, not engine.

Sources

- nightly-labs/openbot on GitHub — repository read at commit `0b311bf` , 2026-11-01